Handling Missing Values

Import Libraries and Data

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly
remount, call drive.mount("/content/drive", force_remount=True).
```

```
dataset =
pd.read_csv('/content/drive/MyDrive/Datsets/Placement_Dataset.csv')
dataset.head()
```


```
{"summary":"{\n  \"name\": \"dataset\",\n  \"rows\": 215,\n
\"fields\": [\n  {\n  \"column\": \"sl_no\",\n
\"properties\": {\n  \"dtype\": \"number\",\n  \"std\":
62,\n  \"min\": 1,\n  \"max\": 215,\n
\"num_unique_values\": 215,\n  \"samples\": [\n  201,\n
213,\n  139\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"gender\",\n  \"properties\": {\n  \"dtype\":
\"category\",\n  \"num_unique_values\": 2,\n  \"samples\":
[\n  \"F\",\n  \"M\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"ssc_p\",\n  \"properties\": {\
n  \"dtype\": \"number\",\n  \"std\": 10.827205398231452,\
n  \"min\": 40.89,\n  \"max\": 89.4,\n
\"num_unique_values\": 103,\n  \"samples\": [\n  74.0,\n
73.96\n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"ssc_b\",\n  \"properties\": {\n  \"dtype\": \"category\",\
n  \"num_unique_values\": 2,\n  \"samples\": [\n
\"Central\",\n  \"Others\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
n  },\n  {\n  \"column\": \"hsc_p\",\n  \"properties\": {\
n  \"dtype\": \"number\",\n  \"std\": 10.89750915750298,\n
\"min\": 37.0,\n  \"max\": 97.7,\n  \"num_unique_values\":
97,\n  \"samples\": [\n  82.0,\n  73.2\
n  ],\n  \"semantic_type\": \"\",\n
\"description\": \"\"\n  }\n  },\n  {\n  \"column\":
\"hsc_b\",\n  \"properties\": {\n  \"dtype\": \"category\",\
n  \"num_unique_values\": 2,\n  \"samples\": [\n
\"Central\",\n  \"Others\"\n  ],\n
\"semantic_type\": \"\",\n  \"description\": \"\"\n  }\
```

n   },\n   {\n   \"column\": \"hsc_s\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 3,\n \"samples\": [\n   \"Commerce\",\n   \"Science\"\n],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n }\n   },\n   {\n   \"column\": \"degree_p\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 7.358743287339439,\n   \"min\": 50.0,\n   \"max\": 91.0,\n \"num_unique_values\": 89,\n   \"samples\": [\n   71.72,\n 76.0\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"degree_t\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 3,\n   \"samples\": [\n   \"Sci&Tech\",\n   \"Comm&Mgmt\"\n   ],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"workex\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"Yes\",\n   \"No\"\n],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n }\n   },\n   {\n   \"column\": \"etest_p\",\n \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 13.275956401653835,\n   \"min\": 50.0,\n   \"max\": 98.0,\n \"num_unique_values\": 100,\n   \"samples\": [\n   93.4,\n 69.0\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"specialisation\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"Mkt&Fin\",\n   \"Mkt&HR\"\n   ],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"mba_p\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 5.8333845806838,\n \"min\": 51.21,\n   \"max\": 77.89,\n \"num_unique_values\": 205,\n   \"samples\": [\n   64.66,\n   52.21\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"status\",\n   \"properties\": {\n   \"dtype\": \"category\",\n   \"num_unique_values\": 2,\n   \"samples\": [\n   \"Not Placed\",\n   \"Placed\"\n   ],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"salary\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 93457.45241958875,\n   \"min\": 200000.0,\n   \"max\": 940000.0,\n   \"num_unique_values\": 45,\n   \"samples\": [\n   255000.0,\n   336000.0\n   ],\n \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"dataset"}

```
dataset.shape
```

```
(215, 15)
```

```
dataset.isnull().sum()
```

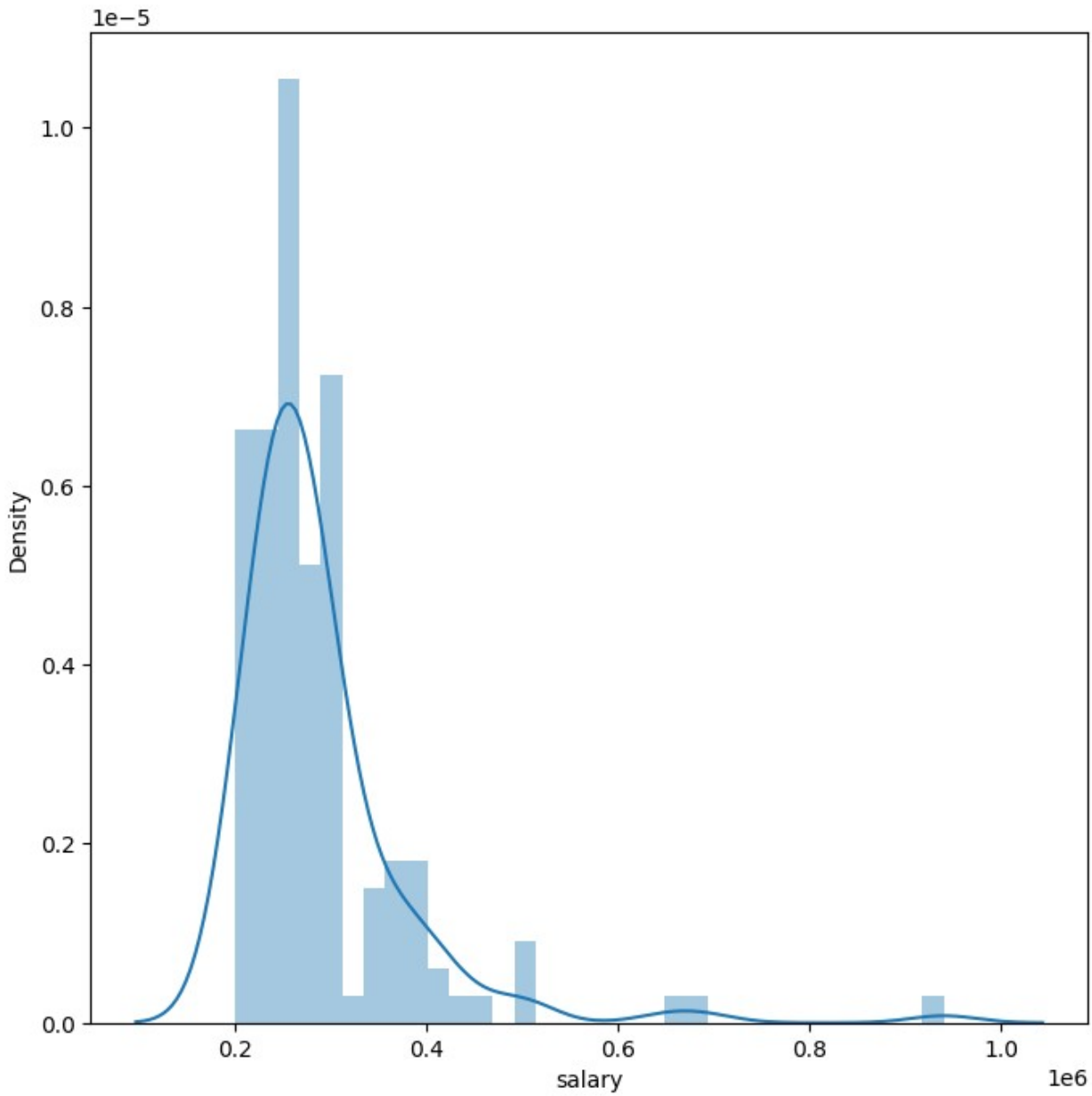
```
sl_no               0
gender              0
ssc_p               0
ssc_b               0
hsc_p               0
hsc_b               0
hsc_s               0
degree_p            0
degree_t            0
workex              0
etest_p             0
specialisation      0
mba_p               0
status              0
salary             67
dtype: int64
```

Central Tendencies

```
# analyse the distribution of data in the salary
fig, ax = plt.subplots(figsize=(8,8))
sns.distplot(dataset.salary)
```

```
/tmp/ipython-input-1844316234.py:3: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn
v0.14.0.

Please adapt your code to use either `displot` (a figure-level
function with
similar flexibility) or `histplot` (an axes-level function for
histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(dataset.salary)
```

```
<Axes: xlabel='salary', ylabel='Density'>
```




```
# Replacing missing values with Median values
dataset.fillna({'salary': dataset['salary'].median()}, inplace=True)

dataset.isnull().sum()
```

```
sl_no             0
gender            0
ssc_p             0
ssc_b             0
hsc_p             0
hsc_b             0
hsc_s             0
```

```
degree_p          0
degree_t          0
workex            0
etest_p           0
specialisation    0
mba_p             0
status            0
salary            0
dtype: int64
```

```
# filling missing values with Mean value:
dataset['salary'].fillna(dataset['salary'].mean(),inplace=True)
# filling missing values with Mean value:
dataset['salary'].fillna(dataset['salary'].mode(),inplace=True)
```

```
/tmp/ipython-input-114496927.py:2: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  dataset['salary'].fillna(dataset['salary'].mean(),inplace=True)
/tmp/ipython-input-114496927.py:4: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  dataset['salary'].fillna(dataset['salary'].mode(),inplace=True)
```

```
# Dropping Method
salary_dataset =
pd.read_csv('/content/drive/MyDrive/Datsets/Placement_Dataset.csv')
salary_dataset.shape
```

```
(215, 15)
```

```
# drop the missing values
salary_dataset = salary_dataset.dropna(how='any')
salary_dataset.shape
```

```
(148, 15)
```

2. Data Standardization:

```
import numpy as np
import pandas as pd
import sklearn.datasets
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# loading the dataset
dataset = sklearn.datasets.load_breast_cancer()
print(dataset)
```

```
{'data': array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01,
4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0,
0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0,
0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1,
0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1,
1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0,
```

0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0,
0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0,
0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1,
0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1,
1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1]),
'frame': None, 'target_names': array(['malignant', 'benign'],
dtype='<U9'), 'DESCR': '.. _breast_cancer_dataset:\n\nBreast cancer
wisconsin (diagnostic) dataset\
n--\n\n**Data Set
Characteristics:**\n\n:Number of Instances: 569\n\n:Number of
Attributes: 30 numeric, predictive attributes and the class\n\
n:Attribute Information:\n   - radius (mean of distances from center
to points on the perimeter)\n   - texture (standard deviation of
gray-scale values)\n   - perimeter\n   - area\n   - smoothness
(local variation in radius lengths)\n   - compactness (perimeter^2 /
area - 1.0)\n   - concavity (severity of concave portions of the
contour)\n   - concave points (number of concave portions of the
contour)\n   - symmetry\n   - fractal dimension ("coastline
approximation" - 1)\n\n   The mean, standard error, and "worst" or
largest (mean of the three\n   worst/largest values) of these

features were computed for each image,\n   resulting in 30 features. For instance, field 0 is Mean Radius, field\n   10 is Radius SE, field 20 is Worst Radius.\n\n   - class:\n   - WDBC-Malignant\n   - WDBC-Benign\n\n:Summary Statistics:\n\n===================================== ====== ======\n   Min   Max\n===================================== ====== ======\nradius (mean):   6.981 28.11\ntexture (mean):   9.71   39.28\nperimeter (mean):   43.79 188.5\narea (mean):   143.5 2501.0\nsmoothness (mean):   0.053 0.163\ncompactness (mean):   0.019 0.345\nconcavity (mean):   0.0   0.427\nconcave points (mean):   0.0   0.201\nsymmetry (mean):   0.106 0.304\nfractal dimension (mean):   0.05   0.097\nradius (standard error):   0.112 2.873\ntexture (standard error):   0.36   4.885\nperimeter (standard error):   0.757 21.98\narea (standard error):   6.802 542.2\nsmoothness (standard error):   0.002 0.031\ncompactness (standard error):   0.002 0.135\nconcavity (standard error):   0.0   0.396\nconcave points (standard error):   0.0   0.053\nsymmetry (standard error):   0.008 0.079\nfractal dimension (standard error):   0.001 0.03\nradius (worst):   7.93   36.04\ntexture (worst):   12.02 49.54\nperimeter (worst):   50.41 251.2\narea (worst):   185.2 4254.0\nsmoothness (worst):   0.071 0.223\ncompactness (worst):   0.027 1.058\nconcavity (worst):   0.0   1.252\nconcave points (worst):   0.0   0.291\nsymmetry (worst):   0.156 0.664\nfractal dimension (worst):   0.055 0.208\n===================================== ====== ======\n\n:Missing Attribute Values: None\n\n:Class Distribution: 212 - Malignant, 357 - Benign\n\n:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L. Mangasarian\n\n:Donor: Nick Street\n\n:Date: November, 1995\n\nThis is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.\nhttps://goo.gl/U2Uwz2\n\nFeatures are computed from a digitized image of a fine needle\naspirate (FNA) of a breast mass. They describe\ncharacteristics of the cell nuclei present in the image.\n\nSeparating plane described above was obtained using\nMultisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree\nConstruction Via Linear Programming." Proceedings of the 4th\nMidwest Artificial Intelligence and Cognitive Science Society,\npp. 97-101, 1992], a classification method which uses linear\nprogramming to construct a decision tree. Relevant features\nwere selected using an exhaustive search in the space of 1-4\nfeatures and 1-3 separating planes.\n\nThe actual linear program used to obtain the separating plane\nin the 3-dimensional space is that described in:\n[K. P. Bennett and O. L. Mangasarian: "Robust Linear\nProgramming Discrimination of Two Linearly Inseparable Sets",\nOptimization Methods and Software 1, 1992, 23-34].\n\nThis database is also available through the UW CS ftp server:\n\nftp ftp.cs.wisc.edu\ncd math-prog/cpo-dataset/machine-

```
learn/WDBC/\n\n.. dropdown:: References\n\n  - W.N. Street, W.H.
Wolberg and O.L. Mangasarian. Nuclear feature extraction\n    for
breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on\n
Electronic Imaging: Science and Technology, volume 1905, pages 861-
870,\n    San Jose, CA, 1993.\n  - O.L. Mangasarian, W.N. Street and
W.H. Wolberg. Breast cancer diagnosis and\n    prognosis via linear
programming. Operations Research, 43(4), pages 570-577,\n    July-
August 1995.\n  - W.H. Wolberg, W.N. Street, and O.L. Mangasarian.
Machine learning techniques\n    to diagnose breast cancer from fine-
needle aspirates. Cancer Letters 77 (1994)\n    163-171.\n',
'feature_names': array(['mean radius', 'mean texture', 'mean
perimeter', 'mean area',
       'mean smoothness', 'mean compactness', 'mean concavity',
       'mean concave points', 'mean symmetry', 'mean fractal
dimension',
       'radius error', 'texture error', 'perimeter error', 'area
error',
       'smoothness error', 'compactness error', 'concavity error',
       'concave points error', 'symmetry error',
       'fractal dimension error', 'worst radius', 'worst texture',
       'worst perimeter', 'worst area', 'worst smoothness',
       'worst compactness', 'worst concavity', 'worst concave points',
       'worst symmetry', 'worst fractal dimension'], dtype='<U23'),
'filename': 'breast_cancer.csv', 'data_module':
'sklearn.datasets.data'}
```

```
# loading the data to a pandas dataframe
df = pd.DataFrame(dataset.data, columns=dataset.feature_names)
df.head()
```

{"type":"dataframe","variable_name":"df"}

```
X = df
Y = dataset.target
```

```
X
```

{"type":"dataframe","variable_name":"df"}

```
Y
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1,
1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0,
0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0,
0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0,
1,
```

```
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1,
0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1,
1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0,
0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0,
0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1,
1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0,
0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1,
1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1,
0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0,
0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1,
1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1])
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size=0.2)
print(X.shape, X_train.shape, X_test.shape)
```

```
(569, 30) (455, 30) (114, 30)
```

```
# Standardize the data
print(dataset.data.std())
```

```
scaler = StandardScaler()
scaler.fit(X_train)
X_train_standardized = scaler.transform(X_train)
X_test_standardized = scaler.transform(X_test)
```

```
228.29740508276657
```

```
print(print(X_train_standardized))
```

```
[[ 0.19807559  0.7701116   0.39266046 ...  1.98045482 -0.06767815
   1.67669895]
 [-1.51877019 -0.80188674 -1.47772378 ... -0.16672871  0.03043738
   0.93402847]
 [-0.74182378 -0.11677702 -0.77016381 ... -0.79717193  0.65183567
  -0.75258694]
 ...
 [-0.49989702 -0.41475282 -0.47501022 ... -0.27160167 -0.1445353
   1.47757716]
 [ 0.93219817  0.01696015  0.91827644 ...  1.17281246  0.03370789
  -0.30537016]
 [-0.34695482 -0.72211369 -0.37231295 ... -0.48496391  0.63711834
  -0.32312967]]
None
```

```
print(X_train_standardized.std())
print(X_test_standardized.std())
```

```
1.0
0.9805350933683704
```

Label Encoding

```
# importing the Dependencies
import pandas as pd
from sklearn.preprocessing import LabelEncoder

# loading the data from csv file to pandas dataFrame
cancer_data =
pd.read_csv('/content/drive/MyDrive/Datsets/Breast_Cancer_Dataset.csv'
)
# first 5 rows of the dataframe
cancer_data.head()
```

{"type":"dataframe","variable_name":"cancer_data"}

```
# finding the count of different labels
cancer_data['diagnosis'].value_counts()
```

```
diagnosis
B    357
```

```
M    212
Name: count, dtype: int64
```

```
# load the Label Encoder function
label_encode = LabelEncoder()
labels = label_encode.fit_transform(cancer_data.diagnosis)
# appending the labels to the DataFrame
cancer_data['target'] = labels
cancer_data.head()
```

{"type":"dataframe","variable_name":"cancer_data"}

```
cancer_data['target'].value_counts()
# 0 --> Benign
# 1 --> Malignant
```

```
target
0    357
1    212
Name: count, dtype: int64
```

Label Encoding of iris data

```
# loading the data from csv file to pandas dataFrame
iris_data =
pd.read_csv('/content/drive/MyDrive/Datsets/iris_data.csv')
iris_data.head()
```

{"summary":"{\n \"name\": \"iris_data\",\n \"rows\": 150,\n \"fields\": [\n {\n \"column\": \"Id\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 43,\n \"min\": 1,\n \"max\": 150,\n \"num_unique_values\": 150,\n \"samples\": [\n 74,\n 19,\n 119\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"SepalLengthCm\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.8280661279778629,\n \"min\": 4.3,\n \"max\": 7.9,\n \"num_unique_values\": 35,\n \"samples\": [\n 6.2,\n 4.5,\n 5.6\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"SepalWidthCm\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.4335943113621737,\n \"min\": 2.0,\n \"max\": 4.4,\n \"num_unique_values\": 23,\n \"samples\": [\n 2.3,\n 4.0,\n 3.5\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"PetalLengthCm\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1.7644204199522617,\n \"min\": 1.0,\n \"max\": 6.9,\n \"num_unique_values\": 43,\n \"samples\": [\n 6.7,\n 3.8,\n 3.7\n

],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"PetalWidthCm\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.7631607417008414,\n   \"min\": 0.1,\n   \"max\": 2.5,\n
\"num_unique_values\": 22,\n   \"samples\": [\n   0.2,\n
1.2,\n   1.3\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Species\",\n   \"properties\": {\n   \"dtype\":
\"category\",\n   \"num_unique_values\": 3,\n   \"samples\":
[\n   \"Iris-setosa\",\n   \"Iris-versicolor\",\n
\"Iris-virginica\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"iris_data"}

```
iris_data['Species'].value_counts()
```

```
Species
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

```
# loding the label encoder
label_encoder_1 = LabelEncoder()
iris_labels = label_encoder_1.fit_transform(iris_data.Species)
iris_data['target'] = iris_labels
iris_data.head()
```

{"summary":"{\n \"name\": \"iris_data\",\n \"rows\": 150,\n
\"fields\": [\n   {\n   \"column\": \"Id\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 43,\n
\"min\": 1,\n   \"max\": 150,\n   \"num_unique_values\":
150,\n   \"samples\": [\n   74,\n   19,\n
119\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"SepalLengthCm\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.8280661279778629,\n   \"min\":
4.3,\n   \"max\": 7.9,\n   \"num_unique_values\": 35,\n
\"samples\": [\n   6.2,\n   4.5,\n   5.6\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"SepalWidthCm\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.4335943113621737,\n   \"min\": 2.0,\n   \"max\": 4.4,\n
\"num_unique_values\": 23,\n   \"samples\": [\n   2.3,\n
4.0,\n   3.5\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"PetalLengthCm\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 1.7644204199522617,\n   \"min\":
1.0,\n   \"max\": 6.9,\n   \"num_unique_values\": 43,\n
\"samples\": [\n   6.7,\n   3.8,\n   3.7\n

],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"PetalWidthCm\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.7631607417008414,\n   \"min\": 0.1,\n   \"max\": 2.5,\n
\"num_unique_values\": 22,\n   \"samples\": [\n   0.2,\n
1.2,\n   1.3\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Species\",\n   \"properties\": {\n   \"dtype\":
\"category\",\n   \"num_unique_values\": 3,\n   \"samples\":
[\n   \"Iris-setosa\",\n   \"Iris-versicolor\",\n
\"Iris-virginica\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"target\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0,\n   \"min\": 0,\n   \"max\": 2,\n
\"num_unique_values\": 3,\n   \"samples\": [\n   0,\n
1,\n   2\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"iris_data"}

```
iris_data['target'].value_counts()

# Iris-setosa --> 0
# Iris-versicolor --> 1
# Iris-virginica --> 2
```

```
target
0    50
1    50
2    50
Name: count, dtype: int64
```

4. Train Test Split

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score

# loading the PIMA Diabetes Dataset to a pandas DataFrame
diabetes_dataset =
pd.read_csv('/content/drive/MyDrive/Datsets/diabetes.csv')
# printing the first 5 rows of the dataset
diabetes_dataset.head()
```

{"summary":"{\n \"name\": \"diabetes_dataset\",\n \"rows\": 768,\n
\"fields\": [\n   {\n   \"column\": \"Pregnancies\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":

3,\n   \"min\": 0,\n   \"max\": 17,\n   \"num_unique_values\": 17,\n   \"samples\": [\n   6,\n   1,\n   3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Glucose\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 31,\n   \"min\": 0,\n   \"max\": 199,\n   \"num_unique_values\": 136,\n   \"samples\": [\n   151,\n   101,\n   112\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BloodPressure\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 19,\n   \"min\": 0,\n   \"max\": 122,\n   \"num_unique_values\": 47,\n   \"samples\": [\n   86,\n   46,\n   85\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"SkinThickness\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 15,\n   \"min\": 0,\n   \"max\": 99,\n   \"num_unique_values\": 51,\n   \"samples\": [\n   7,\n   12,\n   48\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Insulin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 115,\n   \"min\": 0,\n   \"max\": 846,\n   \"num_unique_values\": 186,\n   \"samples\": [\n   52,\n   41,\n   183\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BMI\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 7.8841603203754405,\n   \"min\": 0.0,\n   \"max\": 67.1,\n   \"num_unique_values\": 248,\n   \"samples\": [\n   19.9,\n   31.0,\n   38.1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"DiabetesPedigreeFunction\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.33132859501277484,\n   \"min\": 0.078,\n   \"max\": 2.42,\n   \"num_unique_values\": 517,\n   \"samples\": [\n   1.731,\n   0.426,\n   0.138\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Age\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 11,\n   \"min\": 21,\n   \"max\": 81,\n   \"num_unique_values\": 52,\n   \"samples\": [\n   60,\n   47,\n   72\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Outcome\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0,\n   \"min\": 0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   0,\n   1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"diabetes_dataset"}

```
# number of rows and Columns in this dataset
diabetes_dataset.shape
```

```
(768, 9)
```

```
# getting the statistical measures of the data
diabetes_dataset.describe()
```


```
{"summary":"{\n  \"name\": \"diabetes_dataset\",\n  \"rows\": 8,\n  \"fields\": [\n  {\n  \"column\": \"Pregnancies\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 269.85223453356366,\n  \"min\": 0.0,\n  \"max\": 768.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  3.8450520833333335,\n  3.0,\n  768.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Glucose\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 243.73802348295857,\n  \"min\": 0.0,\n  \"max\": 768.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  120.89453125,\n  117.0,\n  768.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"BloodPressure\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 252.85250535810619,\n  \"min\": 0.0,\n  \"max\": 768.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  69.10546875,\n  72.0,\n  768.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"SkinThickness\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 263.7684730531098,\n  \"min\": 0.0,\n  \"max\": 768.0,\n  \"num_unique_values\": 7,\n  \"samples\": [\n  768.0,\n  20.536458333333332,\n  32.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"Insulin\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 350.26059167945886,\n  \"min\": 0.0,\n  \"max\": 846.0,\n  \"num_unique_values\": 7,\n  \"samples\": [\n  768.0,\n  79.79947916666667,\n  127.25\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"BMI\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 262.05117817552093,\n  \"min\": 0.0,\n  \"max\": 768.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  31.992578124999998,\n  32.0,\n  768.0\n  ],\n  \"semantic_type\": \"\",\n  \"description\": \"\"\n  }\n  },\n  {\n  \"column\": \"DiabetesPedigreeFunction\",\n  \"properties\": {\n  \"dtype\": \"number\",\n  \"std\": 271.3005221658502,\n  \"min\": 0.078,\n  \"max\": 768.0,\n  \"num_unique_values\": 8,\n  \"samples\": [\n  0.47187630208333325,\n  0.3725,\n  768.0\n  ],\n
```

\"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Age\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 260.1941178528413,\n \"min\": 11.76023154067868,\n \"max\": 768.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 33.240885416666664,\n 29.0,\n 768.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Outcome\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 271.3865920388932,\n \"min\": 0.0,\n \"max\": 768.0,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.3489583333333333,\n 1.0,\n 0.4769513772427971\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

```
diabetes_dataset['Outcome'].value_counts()
# 0 --> Non-Diabetic
# 1 --> Diabetic
```

```
Outcome
0    500
1    268
Name: count, dtype: int64
```

```
diabetes_dataset.groupby('Outcome').mean()
```

{"summary":"{\n \"name\": \"diabetes_dataset\",\n \"rows\": 2,\n \"fields\": [\n {\n \"column\": \"Outcome\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0,\n \"min\": 0,\n \"max\": 1,\n \"num_unique_values\": 2,\n \"samples\": [\n 1,\n 0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Pregnancies\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1.108511248584296,\n \"min\": 3.298,\n \"max\": 4.865671641791045,\n \"num_unique_values\": 2,\n \"samples\": [\n 4.865671641791045,\n 3.298\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Glucose\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 22.116505963980842,\n \"min\": 109.98,\n \"max\": 141.25746268656715,\n \"num_unique_values\": 2,\n \"samples\": [\n 141.25746268656715,\n 109.98\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"BloodPressure\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 1.8672051632998017,\n \"min\": 68.184,\n \"max\": 70.82462686567165,\n \"num_unique_values\": 2,\n \"samples\": [\n 70.82462686567165,\n 68.184\n

],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"SkinThickness\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
1.7678935989570275,\n   \"min\": 19.664,\n   \"max\":
22.16417910447761,\n   \"num_unique_values\": 2,\n
\"samples\": [\n   22.16417910447761,\n   19.664\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"Insulin\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
22.304849659757796,\n   \"min\": 68.792,\n   \"max\":
100.33582089552239,\n   \"num_unique_values\": 2,\n
\"samples\": [\n   100.33582089552239,\n   68.792\n
],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n
}\n   },\n   {\n   \"column\": \"BMI\",\n   \"properties\": {\
n   \"dtype\": \"number\",\n   \"std\": 3.4212211239962618,\
n   \"min\": 30.3042,\n   \"max\": 35.14253731343284,\n
\"num_unique_values\": 2,\n   \"samples\": [\n
35.14253731343284,\n   30.3042\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"DiabetesPedigreeFunction\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.08539445753677459,\n   \"min\": 0.429734,\n   \"max\":
0.5505,\n   \"num_unique_values\": 2,\n   \"samples\": [\n
0.5505,\n   0.429734\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"Age\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 4.155782645191446,\n   \"min\":
31.19,\n   \"max\": 37.06716417910448,\n
\"num_unique_values\": 2,\n   \"samples\": [\n
37.06716417910448,\n   31.19\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

```
# separating the data and labels
X = diabetes_dataset.drop(columns = 'Outcome', axis=1)
Y = diabetes_dataset['Outcome']
```

```
X
```

{"summary":"{\n \"name\": \"X\",\n \"rows\": 768,\n \"fields\": [\n
{\n   \"column\": \"Pregnancies\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 3,\n   \"min\": 0,\n
\"max\": 17,\n   \"num_unique_values\": 17,\n   \"samples\":
[\n   6,\n   1,\n   3\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Glucose\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 31,\n
\"min\": 0,\n   \"max\": 199,\n   \"num_unique_values\":
136,\n   \"samples\": [\n   151,\n   101,\n
112\n   ],\n   \"semantic_type\": \"\",\n

\"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BloodPressure\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 19,\n   \"min\": 0,\n   \"max\": 122,\n   \"num_unique_values\": 47,\n   \"samples\": [\n   86,\n   46,\n   85\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"SkinThickness\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 15,\n   \"min\": 0,\n   \"max\": 99,\n   \"num_unique_values\": 51,\n   \"samples\": [\n   7,\n   12,\n   48\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Insulin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 115,\n   \"min\": 0,\n   \"max\": 846,\n   \"num_unique_values\": 186,\n   \"samples\": [\n   52,\n   41,\n   183\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BMI\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 7.8841603203754405,\n   \"min\": 0.0,\n   \"max\": 67.1,\n   \"num_unique_values\": 248,\n   \"samples\": [\n   19.9,\n   31.0,\n   38.1\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"DiabetesPedigreeFunction\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.33132859501277484,\n   \"min\": 0.078,\n   \"max\": 2.42,\n   \"num_unique_values\": 517,\n   \"samples\": [\n   1.731,\n   0.426,\n   0.138\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Age\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 11,\n   \"min\": 21,\n   \"max\": 81,\n   \"num_unique_values\": 52,\n   \"samples\": [\n   60,\n   47,\n   72\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"X"}

Y

```
0      1
1      0
2      1
3      0
4      1
      ..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

```python
# Data Standardization
scaler = StandardScaler()
scaler.fit(X)
standardized_data = scaler.transform(X)
print(standardized_data)
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
   1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
  -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
  -0.10558415]
 ...
 [ 0.3429808   0.00330087  0.14964075 ... -0.73518964 -0.68519336
  -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
   1.17073215]
 [-0.84488505 -0.8730192   0.04624525 ... -0.20212881 -0.47378505
  -0.87137393]]
```

```python
X = standardized_data
Y = diabetes_dataset['Outcome']

print(X)
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
   1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
  -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
  -0.10558415]
 ...
 [ 0.3429808   0.00330087  0.14964075 ... -0.73518964 -0.68519336
  -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
   1.17073215]
 [-0.84488505 -0.8730192   0.04624525 ... -0.20212881 -0.47378505
  -0.87137393]]
```

```python
# SPLITTING THE DATA INTO TRAINING DATA & TESTING DATA
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size=0.2,random_state=42)
print(X.shape, X_train.shape, X_test.shape)
```

```
(768, 8) (614, 8) (154, 8)
```

5. Handling Imbalanced Dataset

```
# importing the dependencies
import numpy as np
import pandas as pd

# loading the dataset to pandas DataFrame
credit_card_data =
pd.read_csv('/content/drive/MyDrive/Datsets/credit_data.csv')
# first 5 rows of the dataframe
credit_card_data.head()
```

{"type":"dataframe","variable_name":"credit_card_data"}

```
credit_card_data.tail()
```

{"type":"dataframe"}

```
# distribution of the two classes
credit_card_data['Class'].value_counts()

# This is Highly Imbalanced Dataset
# 0 --> Legit Transactions
# 1 --> Fraudulent Transactions
```

```
Class
0    284315
1       492
Name: count, dtype: int64
```

```
# separating the legit and fraudulent transactions
legit = credit_card_data[credit_card_data.Class == 0]
fraud = credit_card_data[credit_card_data.Class == 1]
print(legit.shape)
print(fraud.shape)
```

```
(284315, 31)
(492, 31)
```

```
# Under-sampling
# Build a sample dataset containing similar distribution of Legit &
Fraudulent
# Transactionds

legit_sample = legit.sample(n=492)
print(legit_sample.shape)
```

```
(492, 31)
```

```
# Concatenate the Two DataFrames
new_dataset = pd.concat([legit_sample, fraud], axis = 0)
new_dataset.head()
```

{"type":"dataframe","variable_name":"new_dataset"}

```
new_dataset.tail()
```

{"type":"dataframe"}

```
new_dataset['Class'].value_counts()
```

```
Class
0    492
1    492
Name: count, dtype: int64
```

6. nNumerical_Dataset_Pre_Processing_Use_Case

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Data Collection & Pre-Processing
# loading the data from csv file to a pandas dataframe
diabetes_data =
pd.read_csv('/content/drive/MyDrive/Datsets/diabetes.csv')
# first 5 rows of the dataframe
diabetes_data.head()
```

{"summary":"{\n \"name\": \"diabetes_data\",\n \"rows\": 768,\n
\"fields\": [\n {\n \"column\": \"Pregnancies\",\n
\"properties\": {\n \"dtype\": \"number\",\n \"std\":
3,\n \"min\": 0,\n \"max\": 17,\n
\"num_unique_values\": 17,\n \"samples\": [\n 6,\n
1,\n 3\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\":
\"Glucose\",\n \"properties\": {\n \"dtype\": \"number\",\
n \"std\": 31,\n \"min\": 0,\n \"max\": 199,\n
\"num_unique_values\": 136,\n \"samples\": [\n 151,\n
101,\n 112\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\":
\"BloodPressure\",\n \"properties\": {\n \"dtype\":
\"number\",\n \"std\": 19,\n \"min\": 0,\n
\"max\": 122,\n \"num_unique_values\": 47,\n
\"samples\": [\n 86,\n 46,\n 85\

n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"SkinThickness\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 15,\n   \"min\": 0,\n
\"max\": 99,\n   \"num_unique_values\": 51,\n   \"samples\":
[\n   7,\n   12,\n   48\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Insulin\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 115,\n
\"min\": 0,\n   \"max\": 846,\n   \"num_unique_values\":
186,\n   \"samples\": [\n   52,\n   41,\n
183\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"BMI\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 7.8841603203754405,\n   \"min\": 0.0,\n   \"max\":
67.1,\n   \"num_unique_values\": 248,\n   \"samples\": [\n
19.9,\n   31.0,\n   38.1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"DiabetesPedigreeFunction\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
0.33132859501277484,\n   \"min\": 0.078,\n   \"max\": 2.42,\
n   \"num_unique_values\": 517,\n   \"samples\": [\n
1.731,\n   0.426,\n   0.138\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Age\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 11,\n   \"min\": 21,\n
\"max\": 81,\n   \"num_unique_values\": 52,\n   \"samples\":
[\n   60,\n   47,\n   72\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Outcome\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\": 0,\n
\"min\": 0,\n   \"max\": 1,\n   \"num_unique_values\": 2,\n
\"samples\": [\n   0,\n   1\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe","variable_name":"diabetes_data"}

```
# number of rows & columns
diabetes_data.shape
```

```
(768, 9)
```

```
diabetes_data.describe()
```

{"summary":"{\n \"name\": \"diabetes_data\",\n \"rows\": 8,\n
\"fields\": [\n   {\n   \"column\": \"Pregnancies\",\n
\"properties\": {\n   \"dtype\": \"number\",\n   \"std\":
269.85223453356366,\n   \"min\": 0.0,\n   \"max\": 768.0,\n
\"num_unique_values\": 8,\n   \"samples\": [\n
3.8450520833333335,\n   3.0,\n   768.0\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\

n   },\n   {\n   \"column\": \"Glucose\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 243.73802348295857,\n   \"min\": 0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   120.89453125,\n   117.0,\n   768.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BloodPressure\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 252.85250535810619,\n   \"min\": 0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   69.10546875,\n   72.0,\n   768.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"SkinThickness\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 263.7684730531098,\n   \"min\": 0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 7,\n   \"samples\": [\n   768.0,\n   20.536458333333332,\n   32.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Insulin\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 350.26059167945886,\n   \"min\": 0.0,\n   \"max\": 846.0,\n   \"num_unique_values\": 7,\n   \"samples\": [\n   768.0,\n   79.79947916666667,\n   127.25\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"BMI\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 262.05117817552093,\n   \"min\": 0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   31.992578124999998,\n   32.0,\n   768.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"DiabetesPedigreeFunction\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 271.3005221658502,\n   \"min\": 0.078,\n   \"max\": 768.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   0.47187630208333325,\n   0.3725,\n   768.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Age\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 260.1941178528413,\n   \"min\": 11.76023154067868,\n   \"max\": 768.0,\n   \"num_unique_values\": 8,\n   \"samples\": [\n   33.240885416666664,\n   29.0,\n   768.0\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Outcome\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 271.3865920388932,\n   \"min\": 0.0,\n   \"max\": 768.0,\n   \"num_unique_values\": 5,\n   \"samples\": [\n   0.3489583333333333,\n   1.0,\n   0.4769513772427971\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```python
# Separating Features and Target
X = diabetes_data.drop(columns='Outcome', axis =1)
Y = diabetes_data['Outcome']
print(X)
print(Y)
```

```
     Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI
\
0              6      148             72             35        0  33.6

1              1       85             66             29        0  26.6

2              8      183             64              0        0  23.3

3              1       89             66             23       94  28.1

4              0      137             40             35      168  43.1

..           ...      ...            ...            ...      ...   ...

763           10      101             76             48      180  32.9

764            2      122             70             27        0  36.8

765            5      121             72             23      112  26.2

766            1      126             60              0        0  30.1

767            1       93             70             31        0  30.4


     DiabetesPedigreeFunction  Age
0                       0.627   50
1                       0.351   31
2                       0.672   32
3                       0.167   21
4                       2.288   33
..                        ...  ...
763                     0.171   63
764                     0.340   27
765                     0.245   30
766                     0.349   47
767                     0.315   23

[768 rows x 8 columns]
0      1
1      0
2      1
3      0
4      1
      ..
```

```
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

```
# 0 --> Non - Diabetic
# 1 --> Diabetic

#Data Standardization
scaler = StandardScaler()
standardized_data = scaler.fit_transform(X)
print(standardized_data)
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
   1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
  -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
  -0.10558415]
 ...
 [ 0.3429808   0.00330087  0.14964075 ... -0.73518964 -0.68519336
  -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
   1.17073215]
 [-0.84488505 -0.8730192   0.04624525 ... -0.20212881 -0.47378505
  -0.87137393]]
```

```
X = standardized_data
print(X)
print(Y)
```

```
[[ 0.63994726  0.84832379  0.14964075 ...  0.20401277  0.46849198
   1.4259954 ]
 [-0.84488505 -1.12339636 -0.16054575 ... -0.68442195 -0.36506078
  -0.19067191]
 [ 1.23388019  1.94372388 -0.26394125 ... -1.10325546  0.60439732
  -0.10558415]
 ...
 [ 0.3429808   0.00330087  0.14964075 ... -0.73518964 -0.68519336
  -0.27575966]
 [-0.84488505  0.1597866  -0.47073225 ... -0.24020459 -0.37110101
   1.17073215]
 [-0.84488505 -0.8730192   0.04624525 ... -0.20212881 -0.47378505
  -0.87137393]]
0      1
1      0
2      1
3      0
```

```
4      1
      ..
763    0
764    0
765    0
766    1
767    0
Name: Outcome, Length: 768, dtype: int64
```

```
# Splitting the dataset into Training data & Testing Data
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size=0.2, random_state=42)
print(X.shape, X_train.shape, X_test.shape)
```

```
(768, 8) (614, 8) (154, 8)
```